

Indicaciones Generales

En este laboratorio se trabajará con el conjunto de datos **Male and Female Faces Dataset** (Ashwin Gupta, Kaggle), que contiene aproximadamente 5.4k imágenes de rostros humanos organizadas en dos clases: **male** y **female**. El objetivo es construir, entrenar e interpretar una **red neuronal convolucional (CNN)** desde cero para la clasificación de género, sin emplear modelos preentrenados. Algunos de los ejercicios requieren **investigar e implementar nuevas técnicas de codificación y variación arquitectónica**.

El trabajo combina tres componentes:

- **Trabajo en código:** implementación completa en Python/TensorFlow-Keras.
- **Entregable manuscrito:** descripción de resultados, diagramas explicativos y respuestas analíticas.
- **Presentación en clase:** demostración del modelo funcional desplegado en Streamlit y explicación de los mapas de interpretabilidad.

Ejercicio 1: Descarga y exploración del dataset

Trabajo en código:

- Descargue el dataset desde Kaggle: **Male and Female Faces Dataset** (Ashwin Gupta).
- Organice las carpetas en `data/male/` y `data/female/`.
- Verifique el número de imágenes por clase y explore el tamaño y tipo de archivo de varias imágenes.
- Visualice ejemplos de ambas clases para confirmar la correcta lectura.

Entregable manuscrito:

- Realice una breve descripción del dataset (número de imágenes, clases, variabilidad de tamaños, observaciones generales).
- Incluya un diagrama o esquema que represente la estructura de carpetas y un mosaico de ejemplos representativos.

Ejercicio 2: Preprocesamiento y partición

Trabajo en código:

- Implemente la carga del dataset asegurando que todas las imágenes se mantengan en color (RGB).
- Redimensione todas las imágenes a un tamaño uniforme (por ejemplo, 224x224 píxeles).
- Normalice los valores de los píxeles en el rango $[0,1]$.
- Divida el conjunto en entrenamiento, validación y prueba (por ejemplo, 70/15/15) con una semilla fija.

Entregable manuscrito:

- Describa brevemente el proceso de preprocesamiento.
- Dibuje un esquema del flujo: lectura $\rightarrow$ redimensionamiento $\rightarrow$ normalización $\rightarrow$ partición.
- Explique por qué es importante mantener la consistencia de tamaño y color.

Ejercicio 3: Construcción y entrenamiento del modelo CNN

Trabajo en código:

- Construya una CNN secuencial con al menos tres bloques `Conv2D` + `MaxPooling2D`, seguidos de capas densas.
- Use funciones de activación `ReLU` y una salida sigmoide (`Dense(1, activation='sigmoid')`).
- Compile el modelo con `binary_crossentropy` y el optimizador `Adam`.
- Entrene el modelo y registre las métricas de exactitud y pérdida.
- Guarde el modelo entrenado en `models/model.keras`.

Entregable manuscrito:

- Dibuje la arquitectura de su CNN (capas, filtros, tamaños, activaciones).
- Incluya las curvas de entrenamiento y validación y analice si hay sobreajuste o subajuste.

Ejercicio 4: Ajuste de hiperparámetros

Trabajo en código:

- Realice pequeños experimentos variando hiperparámetros (número de filtros, tamaño del kernel, tasa de aprendizaje o dropout).
- Compare el desempeño de al menos dos configuraciones distintas.

Entregable manuscrito:

- Presente una tabla comparativa con la configuración de cada experimento y sus métricas finales.
- Identifique cuál configuración obtuvo mejor desempeño y justifique su resultado.

Ejercicio 5: Interpretabilidad visual (Saliency Map y Grad-CAM)

Trabajo en código:

- Aplique el método **Saliency Map** para una imagen correctamente clasificada.
- Aplique el método **Grad-CAM** para la misma imagen y una capa convolucional profunda.
- Visualice ambos mapas superpuestos sobre la imagen original.
- Interprete qué regiones o píxeles influyen más en la decisión del modelo.

Entregable manuscrito:

- Explique con palabras propias la diferencia entre los dos métodos.
- Incluya las visualizaciones y un breve análisis de qué zonas del rostro activaron más la red.
- Dibuje o esquematice qué partes del rostro son más relevantes según los mapas.

Ejercicio 6: Despliegue con Streamlit

En esta etapa, el modelo dejará de ser únicamente código en un cuaderno y pasará a convertirse en una **aplicación interactiva**. Para ello se empleará la biblioteca **Streamlit**, una herramienta sencilla y potente que permite construir interfaces gráficas directamente en Python, sin necesidad de conocimientos previos en desarrollo web. Aunque puede que sea la primera vez que trabaje con esta biblioteca, esta actividad representa una excelente oportunidad para aprender a desplegar modelos de **Deep Learning** de forma práctica y accesible. El objetivo final es que el usuario pueda subir una imagen y visualizar tanto la **predicción del modelo** como los **mapas de interpretabilidad** generados por su red neuronal.

Trabajo en código:

- Instale y verifique el correcto funcionamiento de **Streamlit** en su entorno de trabajo.
- Guarde su modelo final (por ejemplo, `model.keras`) y asegúrese de que pueda cargarse correctamente desde otro archivo Python.
- Cree un archivo llamado `streamlit_app.py` que contenga la estructura básica de su aplicación. Esta app deberá:
 1. Permitir al usuario subir una imagen (`st.file_uploader`).
 2. Mostrar la imagen cargada en pantalla.
 3. Preprocesar la imagen de acuerdo con el flujo usado en el entrenamiento del modelo.
 4. Ejecutar la predicción y mostrar la probabilidad de pertenecer a cada clase (`male/female`).
 5. Generar y visualizar los mapas de interpretabilidad (**Grad-CAM** y **Saliency Map**) superpuestos a la imagen.
- Verifique que la aplicación:
 - Cargue el modelo sin errores.
 - Procese correctamente la imagen subida.
 - Muestre resultados de predicción y mapas visuales con claridad.
- El despliegue deberá realizarse en la nube mediante **Streamlit Community Cloud**, siguiendo las instrucciones del **Anexo de Despliegue Web** al final del laboratorio.

Entregable manuscrito:

- Dibuje un esquema de la interfaz de usuario de su aplicación (componentes principales: área de carga, predicción y visualizaciones).
- Describa brevemente el flujo de comunicación entre la interfaz y el modelo (imagen → predicción → mapas).
- Proponga al menos dos mejoras o extensiones posibles.

Ojo: Se evaluará el diseño estético, la **funcionalidad y claridad** del flujo entre el modelo y la interfaz. Verifique que el enlace público funcione correctamente, que los archivos sean accesibles y que las visualizaciones de interpretabilidad sean claras y coherentes.

Ejercicio 7: Presentación y reflexión final

Presentación en clase:

- Demuestre su aplicación funcionando: suba una imagen (propia o de un rostro autorizado), muestre la predicción y explique los mapas de interpretabilidad.
- Interprete.

Evaluación

- Este laboratorio tiene un peso equivalente a **-20 puntos**, si no se entrega completo.
- Se evaluará:
 1. Correctitud, organización y funcionamiento del código.
 2. Claridad y rigor conceptual en el entregable manuscrito.
 3. Calidad y funcionalidad del despliegue en Streamlit.
 4. Capacidad de interpretación durante la presentación en clase.

Entrega final:

- Enlace web funcional de la aplicación desplegada en **Streamlit Cloud**.
- Cuaderno de entrenamiento completo (.ipynb).
- Entregable manuscrito con análisis y diagramas.

Anexo: Despliegue Web en Streamlit Cloud

Este anexo explica cómo publicar su aplicación de clasificación de rostros e interpretabilidad en la web, utilizando la plataforma gratuita **Streamlit Community Cloud**. El resultado será un enlace público (por ejemplo, <https://usuario-proyecto.streamlit.io>) que podrá compartirse directamente con el profesor y el grupo.

1. Crear una cuenta en Streamlit Cloud

- Ingrese a <https://share.streamlit.io>.
- Regístrese con su cuenta de **GitHub** (es necesario para el despliegue).

2. Subir el proyecto a GitHub

- Cree un nuevo repositorio público en su cuenta de GitHub. Asigne un nombre breve, por ejemplo: **cnn-gender-classifier**.
- Suba los siguientes archivos y carpetas:
 - **streamlit_app.py** → archivo principal de la aplicación.
 - **models/model.keras** → modelo entrenado.
 - **requirements.txt** → lista de librerías necesarias.
 - **README.md** → breve descripción (opcional).
- Ejemplo de contenido mínimo de **requirements.txt**:

```
streamlit
tensorflow
numpy
opencv-python
matplotlib
Pillow
```

- Si su modelo pesa más de 100 MB, reduzca su tamaño (menos filtros o capas), ya que Streamlit Cloud no permite archivos mayores a ese límite.

3. Publicar la aplicación

1. Inicie sesión en <https://share.streamlit.io>.
2. Haga clic en “**New app**”.
3. Seleccione el repositorio que contiene su aplicación.
4. En el campo **Main file path**, indique el archivo principal (por ejemplo, `app/streamlit_app.py`).
5. Presione **Deploy**.

El sistema instalará automáticamente las librerías de su `requirements.txt` y lanzará la aplicación en la nube. Este proceso tarda entre 1 y 3 minutos la primera vez.

4. Obtener y verificar el enlace público

- Una vez desplegada, aparecerá una URL similar a: `https://usuario-nombreproyecto.streamlit.app`.
- Verifique que la app:
 - Cargue correctamente el modelo.
 - Permita subir una imagen y mostrar la predicción.
 - Visualice los mapas de interpretabilidad (Grad-CAM y Saliency Map).
- Copie el enlace y colóquelo en la portada de su entregable manuscrito.

6. Recomendaciones finales

- Antes de publicar, pruebe su aplicación localmente con: `streamlit run streamlit_app.py` (opcional si dispone de entorno local).
- Evite rutas absolutas; use siempre rutas relativas dentro de las carpetas del proyecto.
- Revise que las imágenes de salida se muestren con buena resolución y colores coherentes.
- Mantenga el nombre de las carpetas y archivos en minúsculas, sin espacios ni acentos.

Resultado esperado: Cada estudiante tendrá un enlace público activo que permitirá subir imágenes, obtener la predicción de género y visualizar las regiones relevantes del rostro según su modelo CNN.